8-bit Arithmetic Logic Unit (ALU)

Design, RTL Implementation, Verification and FPGA Synthesis

1. Introduction

An Arithmetic Logic Unit (ALU) is a fundamental computational block in a processor datapath. It performs arithmetic, logical, and shift/rotate operations according to a selected opcode. This project presents an 8-bit ALU implemented using synthesizable Verilog-2001 RTL, covering architecture, RTL design, simulation, functional verification, and FPGA-oriented synthesis.

2. Project Objectives

• Design an 8-bit ALU supporting 16 selectable operations.

• Implement clean, synthesizable Verilog-2001 RTL.

• Use a registered datapath with defined pipeline stages and status flags.

• Verify correctness using directed and constrained-random testing.

• Evaluate timing and resource behavior through FPGA synthesis.

• Identify practical improvements for future development.

3. System Specifications

Parameter	Specification
Datapath	8-bit
Opcode	4-bit
Operations	16
HDL	Synthesizable Verilog-2001
Pipeline	2-stage registered datapath
Verification	Self-checking testbench
Target	Xilinx Artix-7 class FPGA / ASIC-portable RTL

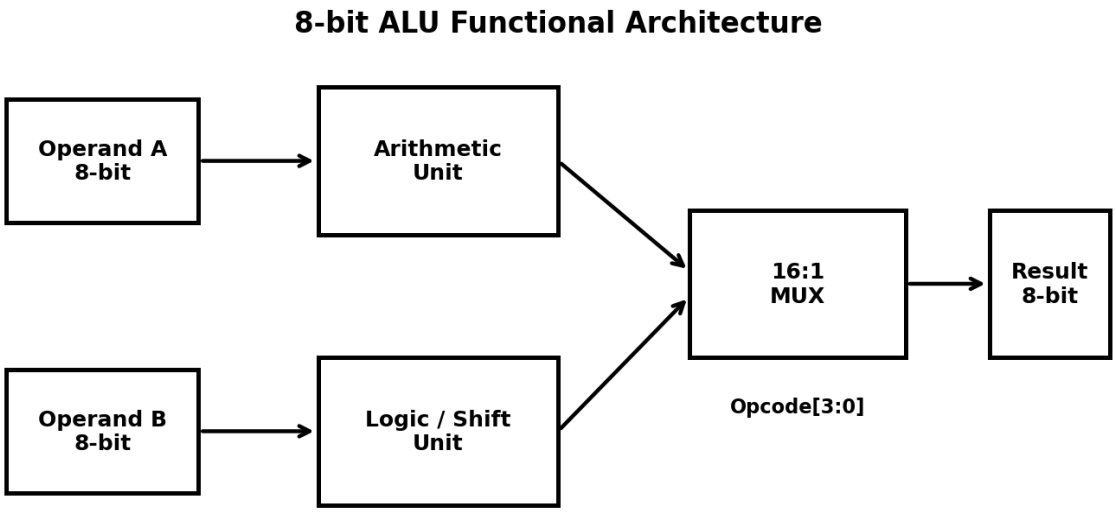

Figure 1 — Simplified 8-bit ALU functional architecture.

4. ALU Architecture

The ALU uses an 8-bit datapath controlled by a 4-bit opcode. Its architecture combines arithmetic, logic, shift/rotate, increment/decrement, and pass-through functions. Parallel functional units feed a result multiplexer, followed by flag generation and registered outputs.

Opcode	Operation	Description
0	ADD	A + B + Cin
1	SUB	A – B
2	AND	A AND B
3	OR	A OR B
4	XOR	A XOR B
5	NOR	NOT (A OR B)
6	NAND	NOT (A AND B)
7	XNOR	NOT (A XOR B)
8	NOT	NOT A
9	SHL	Logical shift left
A	SHR	Logical shift right
B	ROL	Rotate left
C	ROR	Rotate right
D	INC	Increment A
E	DEC	Decrement A
F	PASS	Pass A

5. RTL Design and Functional Units

The RTL datapath is divided into functional units whose outputs are selected by the opcode-driven result multiplexer. This organization gives a clear correspondence between the design specification and synthesized hardware.

5.1 Adder/Subtractor

A 9-bit extended arithmetic path is used for addition and subtraction. Subtraction uses two's-complement addition, allowing arithmetic hardware to be shared.

5.2 Logic Unit

AND, OR, XOR, NOR, NAND, XNOR, and NOT are implemented as bitwise operations on the 8-bit operands.

5.3 Shift and Rotate Unit

Single-bit logical shifts and rotate-left/rotate-right operations use Verilog concatenation and bit selection. The shifted or rotated-out bit is available through the carry output.

5.4 Increment, Decrement and Pass

Increment and decrement provide lightweight arithmetic functions, while PASS provides a direct path for operand A.

6. Flag Generation

Flag	Function
Zero (Z)	Asserted when the registered result equals 00h.
Negative (N)	Taken from the most-significant result bit.
Carry (C)	Uses the ninth arithmetic bit or shifted/rotated-out bit; zero for logical operations.
Overflow (V)	Generated using signed-overflow logic for ADD/SUB and boundary checks for INC/DEC.

7. Simulation and Functional Verification

Behavioral simulation was performed with a self-checking Verilog testbench. Inputs were applied synchronously with the clock and outputs were sampled after the two-cycle pipeline latency. Directed tests targeted important boundary conditions, while constrained-random testing expanded coverage across operands, opcodes, and carry-in.

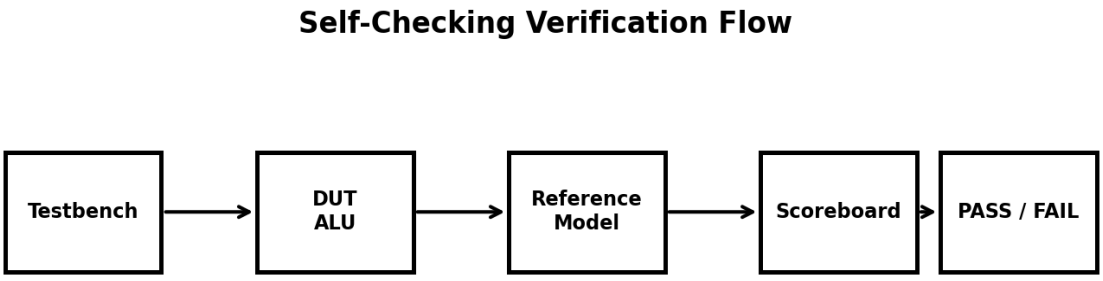

Figure 2 — Simplified self-checking verification flow.

Verification Item	Result
Total checks	264
Mismatches	0
Errors	0
Status	PASS
Simulation time	2.86 µs
Clock reference	100 MHz

The verification suite exercised all 16 opcodes. Boundary tests covered carry, overflow, zero, sign, shift/rotate, increment, and decrement behavior. The self-checking scoreboard compared DUT outputs against an independent reference model; all 264 checks passed with zero mismatches.

8. Hardware Synthesis

The verified RTL was synthesized for a representative Xilinx Artix-7 class FPGA target. The synthesis stage evaluated resource utilization, timing behavior, and estimated power. The technology-agnostic Verilog-2001 implementation can also be considered for ASIC standard-cell implementation.

The reported synthesis results meet the 100 MHz timing target with positive timing margin and use only a small fraction of the available FPGA resources, leaving room for integration into a larger processor datapath.

9. Design Flow

Architecture Specification → Verilog RTL → RTL Review → Functional Simulation → Self-Checking Verification → Synthesis → Timing and Resource Analysis → Future Optimization

10. Future Enhancements

• Parameterize operand width for 16-, 32-, or 64-bit variants.

• Use carry-lookahead or carry-select arithmetic for wider/high-speed designs.

• Add multiplication, division, saturation, and extended shift operations.

• Introduce deeper pipelining when higher clock frequency is required.

- Add design-for-test features for ASIC implementation.
- Complement simulation with formal verification and assertions.
- Retarget the RTL to an ASIC standard-cell library for implementation studies.

11. Conclusion

The project demonstrates the complete design cycle of an 8-bit Arithmetic Logic Unit using synthesizable Verilog-2001 RTL. The ALU supports 16 arithmetic, logical, and shift/rotate operations with carry, overflow, zero, and negative flags. A self-checking verification environment completed 264 checks with zero mismatches, while the reported FPGA synthesis met the 100 MHz timing target and used a small portion of the target device resources. The design provides a practical digital-design building block that can be extended toward a larger processor datapath.